

Programação Orientada a Objetos

Exercícios de 1 a 10 (Herança e polimorfismo)

1 - Crie uma classe `Veiculo` com atributos como modelo e ano. Em seguida, crie duas subclasses, `Carro` e `Moto`, que herdam de `Veiculo`, cada um deve possuir pelo menos 2 atributos e um metodo a mais.

```
public class Veiculo {
    protected String modelo;
    protected int ano;

    public Veiculo(String modelo, int ano) {
        this.modelo = modelo;
        this.ano = ano;
    }
}

public class Carro extends Veiculo {
    private int numeroPortas;
    private String tipoCombustivel;

    public Carro(String modelo, int ano, int numeroPortas, String tipoCombustivel) {
        super(modelo, ano); // Passa os dados para o construtor de Veiculo
        this.numeroPortas = numeroPortas;
        this.tipoCombustivel = tipoCombustivel;
    }

    public void abrirPortaMalas() {
        System.out.println("Porta-malas do " + modelo + " aberto!");
    }
}

public class Moto extends Veiculo {
    private int cilindradas;
    private boolean possuiPartidaEletrica;

    public Moto(String modelo, int ano, int cilindradas, boolean partidaEletrica) {
        super(modelo, ano);
        this.cilindradas = cilindradas;
        this.possuiPartidaEletrica = partidaEletrica;
    }
}
```

```

        public void darGrau() {
            System.out.println("A moto de " + cilindradas + "cc está empinando!");
        }
    }
}

```

2 - Desenvolva uma hierarquia de classes representando animais, começando com uma classe base `Animal` e subclasses como `Mamifero`, `Ave` e `Peixe`

```

public class Animal {

    protected String nome;
    protected int idade;

    // Construtor
    public Animal(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }
}

public class Mamifero extends Animal {

    public Mamifero(String nome, int idade) {
        super(nome, idade); // chama o construtor da classe Animal
    }
}

public class Ave extends Animal {

    public Ave(String nome, int idade) {
        super(nome, idade);
    }
}

public class Peixe extends Animal {

    public Peixe(String nome, int idade) {
        super(nome, idade);
    }
}

```

3 - Implemente uma classe `ContaBancaria` com atributos como saldo e número da conta. Em seguida, crie duas subclasses, `ContaCorrente` e `ContaPoupanca`, que herdam de `ContaBancaria`

```

public class ContaBancaria {
    protected String numeroConta;
    protected double saldo;

    public ContaBancaria(String numeroConta, double saldo) {
        this.numeroConta = numeroConta;
        this.saldo = saldo;
    }

    public void depositar(double valor) {
        this.saldo += valor;
        System.out.println("Depósito de R$"
            + valor
            + " efetuado. Saldo atual: R$"
            + saldo);
    }
}

public class ContaCorrente extends ContaBancaria {
    private double limite;

    public ContaCorrente(String numeroConta, double saldo, double limite)
    {
        super(numeroConta, saldo); // Chama o construtor da ContaBancaria
        this.limite = limite;
    }

    public void sacar(double valor) {
        if (valor <= (saldo + limite)) {
            saldo -= valor;
            System.out.println("Saque de R$" + valor + " realizado.");
        } else {
            System.out.println("Saldo e limite insuficientes.");
        }
    }
}

public class ContaPoupanca extends ContaBancaria {

    public ContaPoupanca(String numeroConta, double saldo) {
        super(numeroConta, saldo);
    }

    public void aplicarRendimento() {
        this.saldo += this.saldo * 0.005; // 0.5% de rendimento
        System.out.println("Rendimento aplicado. Novo saldo: R$" + saldo);
    }
}

```

```
}  
}
```

4 - Utilizando as classes `Carro` e `Moto` do exercício 1, crie um método na classe base `Veiculo` chamado `imprimirInformacoes` que seja sobrescrito nas subclasses para exibir informações específicas de cada veículo

```
public class Veiculo {  
    protected String modelo;  
    protected int ano;  
  
    public Veiculo(String modelo, int ano) {  
        this.modelo = modelo;  
        this.ano = ano;  
    }  
  
    public void imprimirInformacoes() {  
        System.out.println("Modelo: " + modelo + ", Ano: " + ano);  
    }  
}  
  
public class Moto extends Veiculo {  
    private int cilindradas;  
    private String tipo;  
  
    public Moto(String modelo, int ano, int cilindradas, String tipo) {  
        super(modelo, ano);  
        this.cilindradas = cilindradas;  
        this.tipo = tipo;  
    }  
  
    @Override  
    public void imprimirInformacoes() {  
        super.imprimirInformacoes(); // Chama o print de modelo e ano  
        System.out.println(", Cilindradas: " + cilindradas + ", Tipo: " +  
tipo);  
    }  
}  
  
public class Carro extends Veiculo {  
    private String cor;  
    private int portas;  
  
    public Carro(String modelo, int ano, String cor, int portas) {  
        super(modelo, ano);  
        this.cor = cor;  
    }  
}
```

```

        this.portas = portas;
    }

    @Override
    public void imprimirInformacoes() {
        super.imprimirInformacoes(); // Chama o print de modelo e ano
        System.out.println(", Cor: " + cor + ", Portas: " + portas);
    }
}

```

5 - Adicione um método `emitirSom` à classe base `Animal` do exercício 2. Sobrescreva esse método nas subclasses para emitir sons específicos de cada tipo de animal

```

public class Animal {

    protected String nome;
    protected int idade;

    // Construtor
    public Animal(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    // Método comum a todos os animais
    public void emitirSom() {
        System.out.println("O animal faz um som.");
    }
}

public class Mamifero extends Animal {

    public Mamifero(String nome, int idade) {
        super(nome, idade); // chama o construtor da classe Animal
    }

    @Override
    public void emitirSom() {
        System.out.println("O mamífero emite um som.");
    }
}

public class Ave extends Animal {

    public Ave(String nome, int idade) {
        super(nome, idade);
    }
}

```

```

    }

    @Override
    public void emitirSom() {
        System.out.println("A ave canta.");
    }
}

public class Peixe extends Animal {

    public Peixe(String nome, int idade) {
        super(nome, idade);
    }

    @Override
    public void emitirSom() {
        System.out.println("O peixe não faz som audível.");
    }
}

```

6 - Crie um exemplo de polimorfismo por sobrecarga, criando uma classe `Calculadora` que tenha métodos para as 4 operações básicas (soma, subtração, multiplicação, divisão) aceitando diferentes tipos de parâmetros

```

public class Calculadora {

    // ===== SOMA =====
    public int soma(int a, int b) {
        return a + b;
    }

    public double soma(double a, double b) {
        return a + b;
    }

    public double soma(int a, double b) {
        return a + b;
    }

    // ===== SUBTRAÇÃO =====
    public int subtrair(int a, int b) {
        return a - b;
    }

    public double subtrair(double a, double b) {
        return a - b;
    }
}

```

```

    }

    // ===== MULTIPLICAÇÃO =====
    public int multiplicar(int a, int b) {
        return a * b;
    }

    public double multiplicar(double a, double b) {
        return a * b;
    }

    // ===== DIVISÃO =====
    public double dividir(int a, int b) {
        if (b == 0) {
            System.out.println("Erro: divisão por zero.");
            return 0;
        }
        return (double) a / b;
    }

    public double dividir(double a, double b) {
        if (b == 0) {
            System.out.println("Erro: divisão por zero.");
            return 0;
        }
        return a / b;
    }
}

```

7 - Crie uma classe `FormaGeometrica` com métodos para calcular área e perímetro. Em seguida, crie subclasses como `Circulo`, `Retangulo` e `triangulo` que herdam de `FormaGeometrica` e implementam esses métodos

```

public abstract class FormaGeometrica {
    public abstract double calcularArea();
    public abstract double calcularPerimetro();
}

public class Circulo extends FormaGeometrica {

    private double raio;

    public Circulo(double raio) {
        this.raio = raio;
    }
}

```

```

@Override
public double calcularArea() {
    return Math.PI * raio * raio;
}

@Override
public double calcularPerimetro() {
    return 2 * Math.PI * raio;
}
}

public class Retangulo extends FormaGeometrica {

    private double base;
    private double altura;

    public Retangulo(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }

    @Override
    public double calcularArea() {
        return base * altura;
    }

    @Override
    public double calcularPerimetro() {
        return 2 * (base + altura);
    }
}

public class Triangulo extends FormaGeometrica {

    private double lado1;
    private double lado2;
    private double lado3;

    public Triangulo(double lado1, double lado2, double lado3) {
        this.lado1 = lado1;
        this.lado2 = lado2;
        this.lado3 = lado3;
    }

    @Override
    public double calcularPerimetro() {
        return lado1 + lado2 + lado3;
    }
}

```



```

@Override
public double calcularArea() {
    double p = calcularPerimetro() / 2;
    return Math.sqrt(p * (p - lado1) * (p - lado2) * (p - lado3));
}
}

```

8 - Utilize a hierarquia de classes de `animais` do exercício 2 para criar uma lista de animais. Em seguida, percorra a lista chamando o método `emitirSom` de cada animal

```

import java.util.ArrayList;
import java.util.List;

public class Main {

    public static void main(String[] args) {

        // Criando lista de animais
        List<Animal> listaAnimais = new ArrayList<>();

        // Adicionando diferentes tipos de animais
        listaAnimais.add(new Mamifero("Cachorro", 5));
        listaAnimais.add(new Ave("Papagaio", 2));
        listaAnimais.add(new Peixe("Palhaço", 1));
        listaAnimais.add(new Mamifero("Gato", 3));

        // Percorrendo a lista
        System.out.println("=== Sons dos Animais ===");

        for (Animal animal : listaAnimais) {
            System.out.println("Nome: " + animal.nome + ", Idade: " + animal.idade);
            animal.emitirSom(); // Polimorfismo acontece aqui
            System.out.println();
        }
    }
}

```

9 - Expanda a classe `ContaBancaria` do exercício 3 adicionando um método chamado `aplicarJuros`, que é implementado de maneira diferente nas subclasses `ContaCorrente` e `ContaPoupanca`, com valores de 15% na conta corrente e de 6,5% na conta poupança

```

public abstract class ContaBancaria {

    protected String numeroConta;
    protected double saldo;

    public ContaBancaria(String numeroConta, double saldoInicial) {
        this.numeroConta = numeroConta;
        this.saldo = saldoInicial;
    }

    public void depositar(double valor) {
        saldo += valor;
    }

    public void sacar(double valor) {
        if (valor <= saldo) {
            saldo -= valor;
        } else {
            System.out.println("Saldo insuficiente.");
        }
    }

    public double getSaldo() {
        return saldo;
    }

    // Método abstrato
    public abstract void aplicarJuros();
}

public class ContaCorrente extends ContaBancaria {

    private static final double TAXA_JUROS = 0.15;

    public ContaCorrente(String numeroConta, double saldoInicial) {
        super(numeroConta, saldoInicial);
    }

    @Override
    public void aplicarJuros() {
        saldo += saldo * TAXA_JUROS;
        System.out.println("Juros de 15% aplicados na Conta Corrente.");
    }
}

public class ContaPoupanca extends ContaBancaria {

```

```

private static final double TAXA_JUROS = 0.065;

public ContaPoupanca(String numeroConta, double saldoInicial) {
    super(numeroConta, saldoInicial);
}

@Override
public void aplicarJuros() {
    saldo += saldo * TAXA_JUROS;
    System.out.println("Juros de 6,5% aplicados na Conta Poupança.");
}
}

```

10 - Crie uma interface `Autenticavel` com um método `autenticar`. Implemente essa interface nas classes `Usuario` e `Administrador`. Em seguida, crie uma lista de objetos que implementam `Autenticavel` e chame o método `autenticar` para cada um

```

public interface Autenticavel {
    boolean autenticar(String senha);
}

public class Usuario implements Autenticavel {

    private String nome;
    private String senha;

    public Usuario(String nome, String senha) {
        this.nome = nome;
        this.senha = senha;
    }

    @Override
    public boolean autenticar(String senha) {
        if (this.senha.equals(senha)) {
            System.out.println("Usuário " + nome + " autenticado com sucesso.");
            return true;
        } else {
            System.out.println("Falha na autenticação do usuário " + nome);
            return false;
        }
    }
}

public class Administrador implements Autenticavel {

```

```

private String nome;
private String senha;
private int nivelAcesso;

public Administrador(String nome, String senha, int nivelAcesso) {
    this.nome = nome;
    this.senha = senha;
    this.nivelAcesso = nivelAcesso;
}

@Override
public boolean autenticar(String senha) {
    if (this.senha.equals(senha) && nivelAcesso > 0) {
        System.out.println("Administrador " + nome + " autenticado com s
ucesso.");
        return true;
    } else {
        System.out.println("Falha na autenticação do administrador " + n
ome);
        return false;
    }
}
}

```